

Taller Práctico: Preprocesamiento de Datos y Clasificación con Redes Neuronales

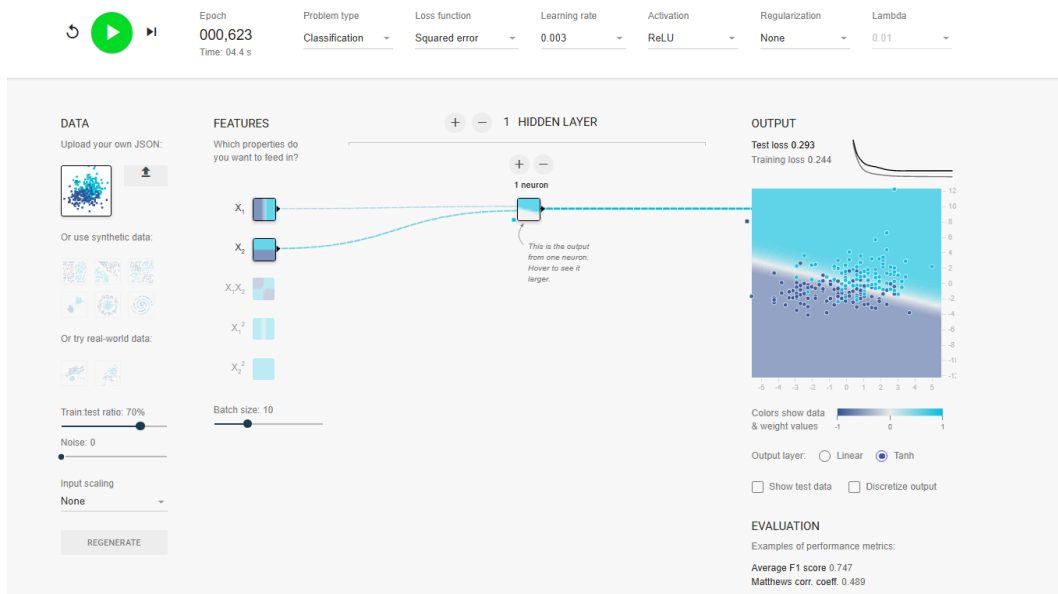
1. Análisis de Experimentos

Para determinar los hiperparámetros y la topología óptima de la Red Neuronal Artificial, se implementó una estrategia de búsqueda sistemática (*Grid Search*). Se evaluaron cinco tasas de aprendizaje (Learning rate: 0.003, 0.01, 0.03, 0.1 y 0.3) contrastadas con diversas arquitecturas, variando tanto la profundidad (1 a 2 capas ocultas) como la amplitud (1 a 4 neuronas por capa).

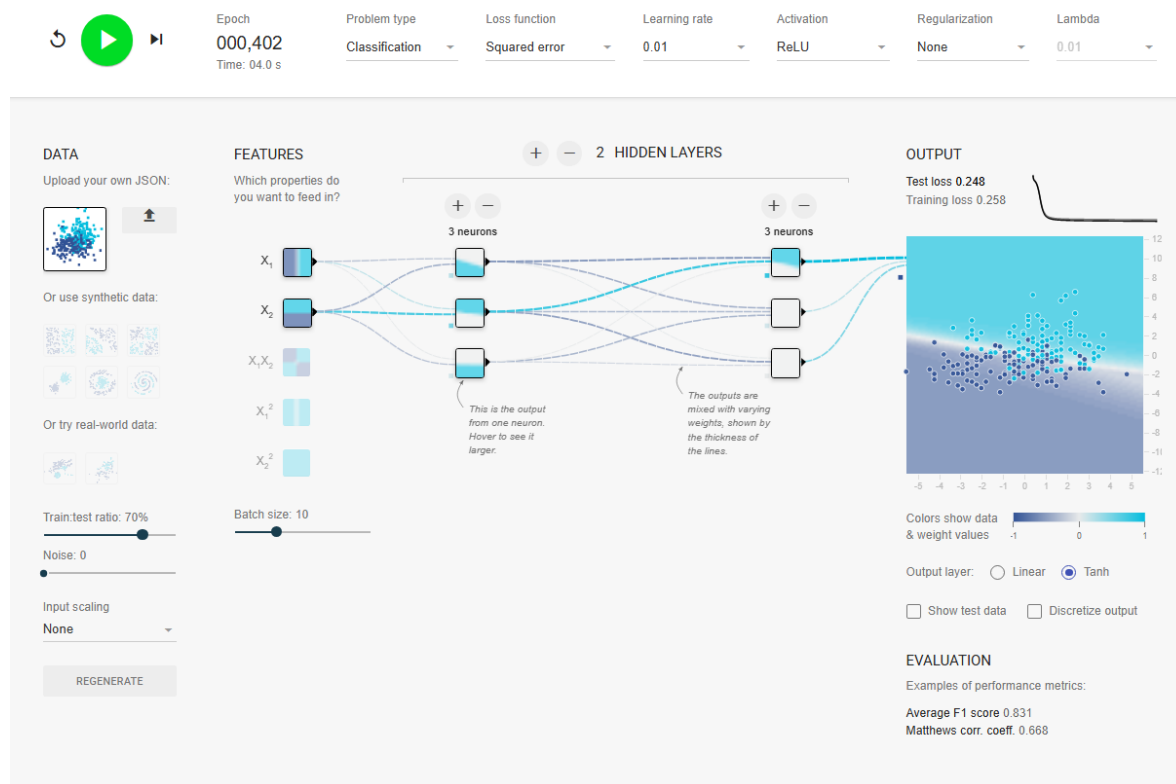
Durante la ejecución, se mantuvo estricto control sobre las variables fijadas por los requerimientos técnicos: función de activación ReLU, función de pérdida *Squared error*, sin regularización y una partición de datos del 70% para entrenamiento (Train) y 30% para validación (Test).

A continuación, se presenta el resumen de los mejores rendimientos obtenidos para cada tasa de aprendizaje evaluada tras la estabilización de la curva de convergencia:

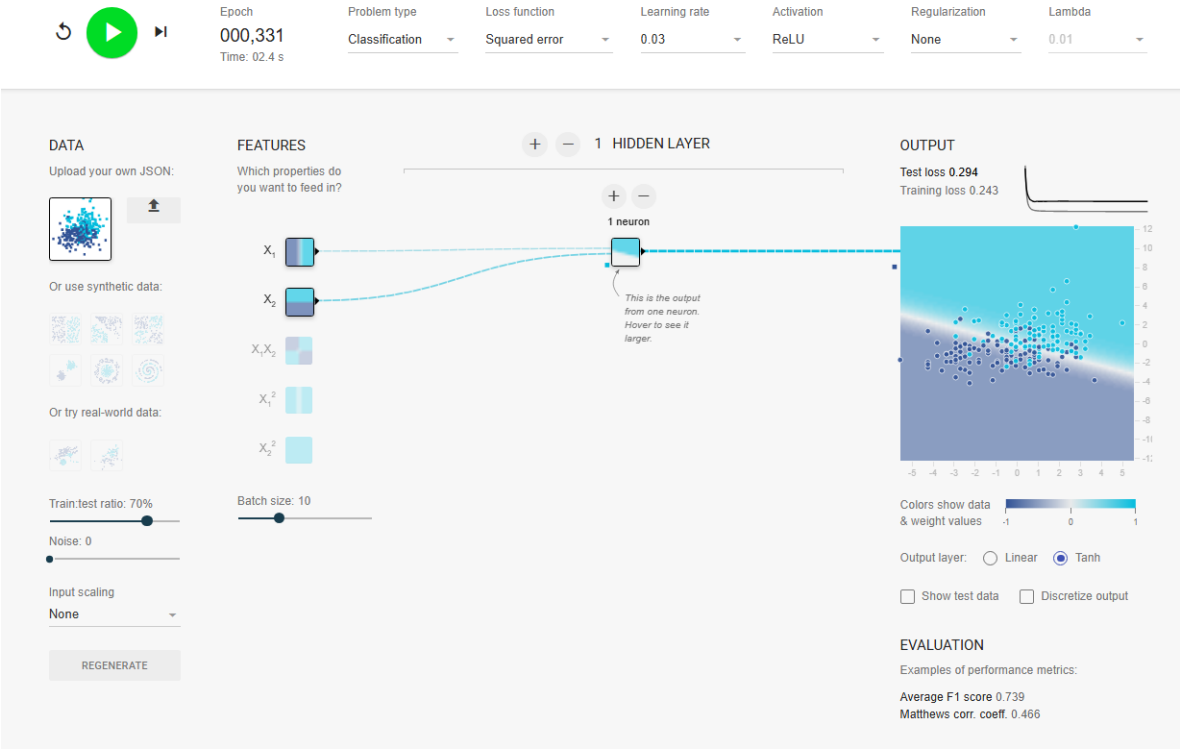
Learning Rate				
0.003				
Arquitectura	Test Loss	Train Loss	F1	Epoch
1 neurona - 1 capa	0.294	0.244	0.747	0.824
2 neuronas - 1 capa	0.343	0.246	0.771	0.786
3 neuronas - 1 capa	0.306	0.236	0.739	0.601
4 neuronas - 1 capa	0.298	0.238	0.723	0.803
3 neuronas - 2 capas	0.332	0.245	0.750	0.433



Learning Rate				
0.01				
Arquitectura	Test Loss	Train Loss	F1	Epoch
1 neurona - 1 capa	0.293	0.243	0.739	0.634
2 neuronas - 1 capa	0.321	0.237	0.739	0.416
3 neuronas - 1 capa	0.324	0.243	0.766	0.557
4 neuronas - 1 capa	0.311	0.235	0.745	0.529
3 neuronas - 2 capas	0.248	0.258	0.831	0.402



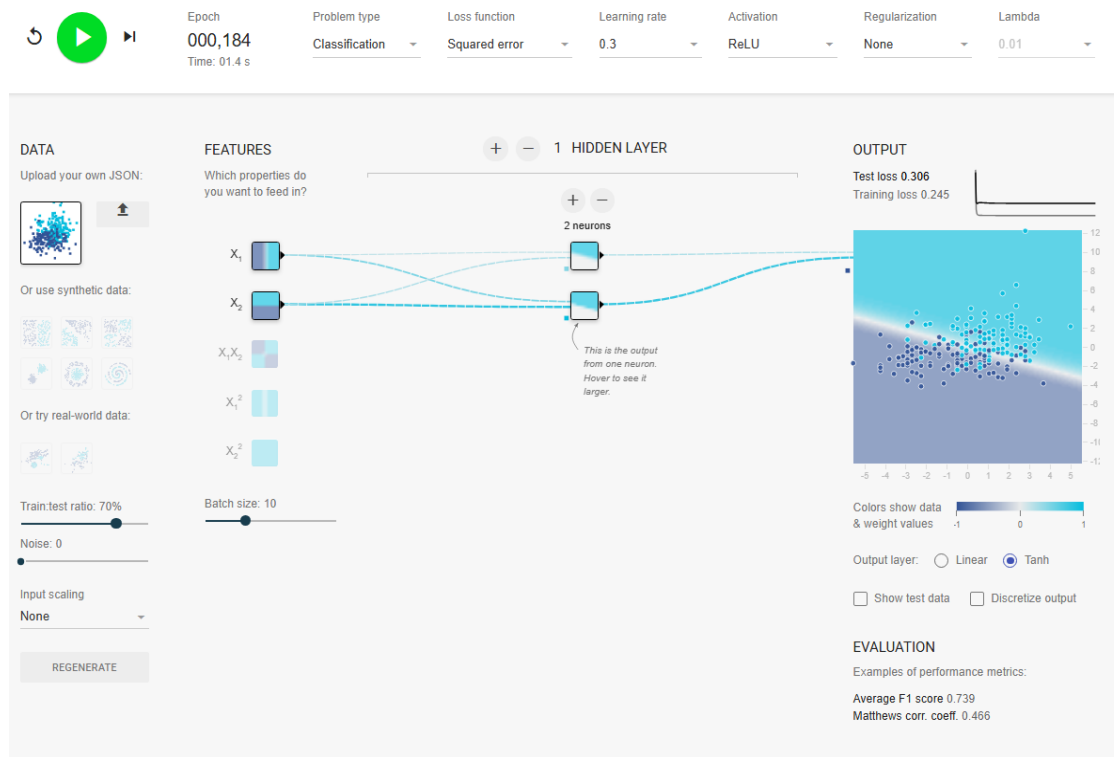
Learning Rate				
0.03				
Arquitectura	Test Loss	Train Loss	F1	Epoch
1 neurona - 1 capa	0.294	0.243	0.739	0.331
2 neuronas - 1 capa	0.309	0.236	0.737	0.498
3 neuronas - 1 capa	0.310	0.236	0.739	0.284
4 neuronas - 1 capa	0.314	0.232	0.723	0.251
3 neuronas - 2 capas	0.307	0.225	0.758	0.667



Learning Rate

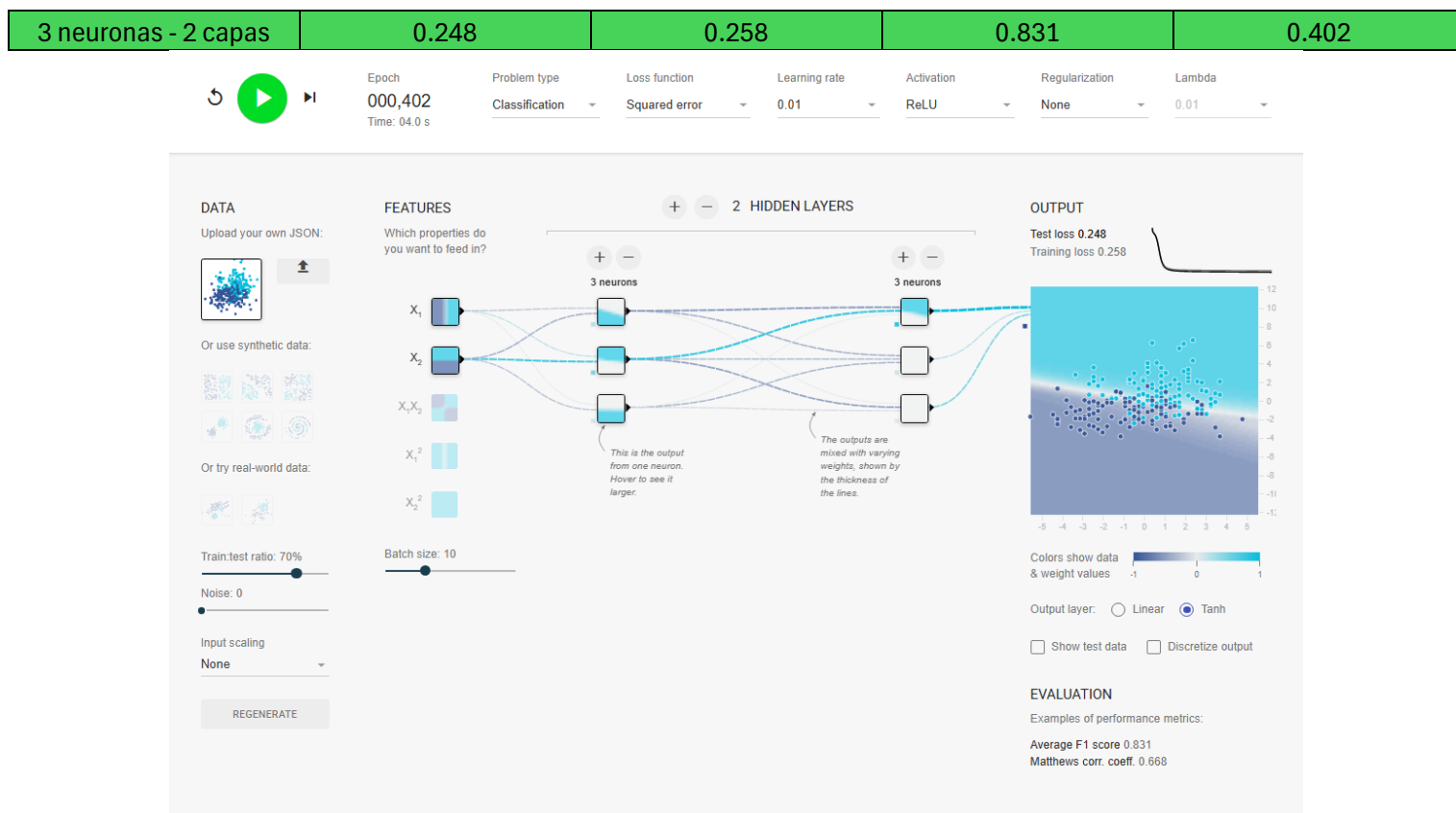
0.3

Arquitectura	Test Loss	Train Loss	F1	Epoch
1 neurona - 1 capa	0.361	0.249	0.739	0.412
2 neuronas - 1 capa	0.306	0.245	0.739	0.184
3 neuronas - 1 capa	0.333	0.243	0.739	0.187
4 neuronas - 1 capa	0.325	0.233	0.723	0.395
3 neuronas - 2 capas	0.308	0.246	0.756	0.300



2. Elección de la Mejor Solución

Configuración seleccionada: La topología óptima corresponde a una arquitectura profunda de 2 capas ocultas con 3 neuronas cada una, operando con un Learning Rate de 0.01.



Justificación Teórica: Desde la perspectiva matemática de la función de pérdida, esta configuración es la solución óptima debido a que logró el mínimo global de error en el conjunto de prueba, registrando un Test loss de 0.248. Al contrastar esta métrica con el rendimiento del conjunto de entrenamiento (Training loss de 0.258), se evidencia una brecha algorítmica mínima de apenas 0.01. Esta simetría en el rendimiento demuestra que el modelo desarrolló una alta capacidad de generalización sin incurrir en memorización de los datos (ausencia de *overfitting*). A nivel espacial, la red neuronal logró procesar correctamente las entradas preprocesadas (X_1 , X_2) para trazar una frontera de decisión no lineal estable, separando eficazmente la clase positiva de la negativa.

3. Código de la Red (Pesos y Biases)

A continuación, se presenta la abstracción en código Python (forward pass) generada por la plataforma, la cual contiene la matriz final de pesos (w) y sesgos tras finalizar el entrenamiento de la configuración óptima:

```
import math

def forward(X1, X2):
    """Compute a forward pass of the network."""
    a1 = max(0, 0.20 + (-0.23 * X1) + (-0.37 * X2))
    a2 = max(0, 0.46 + (0.18 * X1) + (0.68 * X2))
    a3 = max(0, 0.097 + (-0.015 * X1) + (-0.26 * X2))
    a4 = max(0, 0.78 + (-0.54 * a1) + (0.77 * a2) + (0.00069 * a3))
    a5 = max(0, 0.10 + (-0.38 * a1) + (-0.31 * a2) + (-0.34 * a3))
    a6 = max(0, -0.0090 + (0.013 * a1) + (-0.47 * a2) + (-0.12 * a3))
    a7 = math.tanh(-1.1 + (1.1 * a4) + (0.23 * a5) + (0.46 * a6))
    return a7
```

```
import math
```

```
def forward(X1, X2):
```

```
    """Compute a forward pass of the network."""
```

```
    a1 = max(0, 0.20 + (-0.23 * X1) + (-0.37 * X2))
```

```
    a2 = max(0, 0.46 + (0.18 * X1) + (0.68 * X2))
```

```
    a3 = max(0, 0.097 + (-0.015 * X1) + (-0.26 * X2))
```

```
    a4 = max(0, 0.78 + (-0.54 * a1) + (0.77 * a2) + (0.00069 * a3))
```

```
    a5 = max(0, 0.10 + (-0.38 * a1) + (-0.31 * a2) + (-0.34 * a3))
```

```
    a6 = max(0, -0.0090 + (0.013 * a1) + (-0.47 * a2) + (-0.12 * a3))
```

```
    a7 = math.tanh(-1.1 + (1.1 * a4) + (0.23 * a5) + (0.46 * a6))
```

```
    return a7
```